

Perceptron

What is a Perceptron?

Perceptron is a supervised machine learning algorithm and a mathematical model used for binary classification. It can also be considered as a mathematical function.

Components:

- **Inputs ((x))**: Features of the data
 - **Weights ((w))**: Importance assigned to each feature
 - **Bias ((b))**: A constant value added to adjust the weighted sum
 - **Summation ((Σ))**: Weighted sum of inputs and bias
 - **Activation function ((f))**: Converts the weighted sum to a range suitable for classification
-

Perceptron Formula:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

Where:

- (x_i): input features
 - (w_i): corresponding weights
 - (b): bias
 - (f): activation function (e.g., step function, sigmoid)
-

How it works:

- All weights and bias are combined through a dot product with the input features.
 - The sum is then passed through an activation function which maps the output into a specific range.
 - The weights indicate the importance of each feature: a higher weight means that feature is more important in making the prediction.
-

Important Points:

- Perceptron is a **binary classifier**, typically outputting either -1 or 1.
- It works effectively only on **linearly separable** data.

- It fails to converge or perform well on non-linear data.

Common Loss Functions:

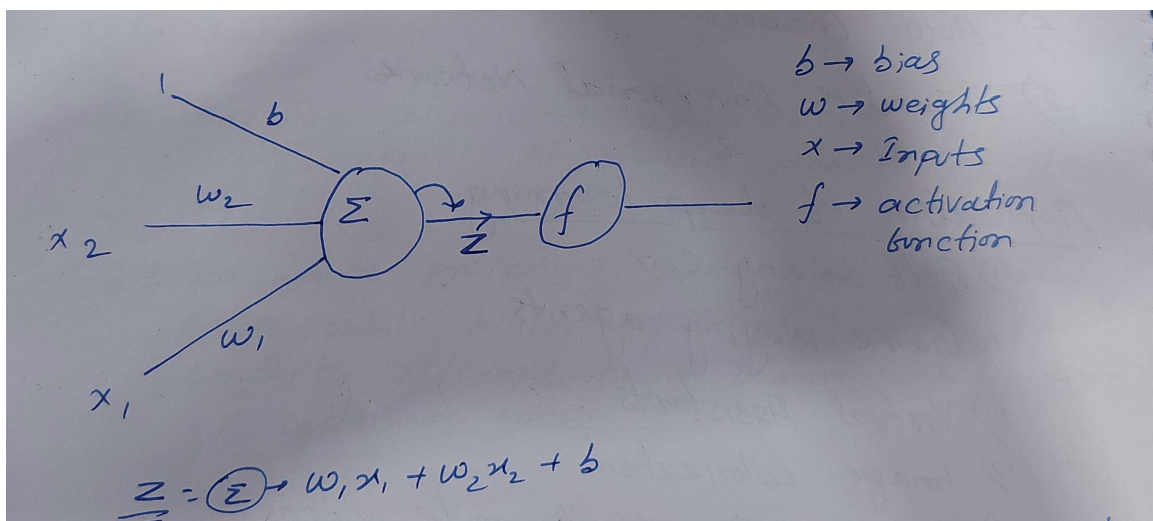
Loss Function	Use Case
Hinge Loss	Perceptron (binary classification)
Cross Entropy	Logistic Regression (binary classification)
Mean Squared Error (MSE)	Regression problems

Activation Functions and Output:

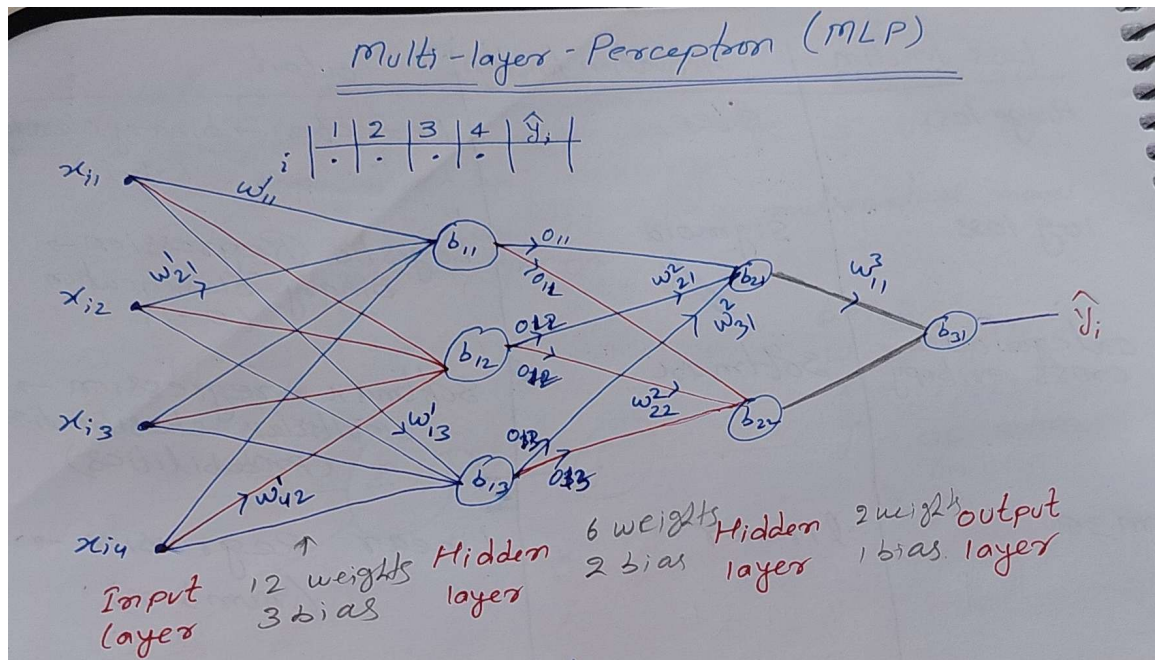
Activation Function	Output Range	Used In
Step Function	Binary (-1, 1)	Perceptron
Sigmoid	Between 0 and 1	Logistic Regression
Softmax	Probabilities for multiple classes	Softmax Regression (multiclass classification)
Linear	Continuous values (real numbers)	Linear Regression

Problem with Perceptron

- Works only on **linearly separable** data.
- Does not converge on **non-linear** data.



Multi-Layer Perceptron (MLP)



- (x_{ix}): Input at row (i), column (x)
 - Here, (i) represents the current row (number of samples/data points)
 - (x) represents the input column (feature index)
 - Example: (x_{13}) means the input feature at row 1, column 3
- Notation for **Trainable Parameters** (Total weights & biases):

$$15 + 8 + 3 = 26$$

 - There are 26 trainable parameters in total (sum of all weights and biases).
- **Notation of Biases:**

$b_{ij} \rightarrow$ Bias of node j in layer i

- **Notation of Output:**

$O_{ij} \rightarrow$ Output from node j in layer i

Notation for Weights

Notation:

$$W_{ij}^{(k)}$$

- (k) $\rightarrow$ The layer into which the node is entering (destination layer)

- **(i)** → Node number in the current (previous) layer (from which the value is coming)
- **(j)** → Node number in the layer (k) (to which the value is entering)

This notation helps identify:

- **From which node** in the previous layer a signal is coming.
- **To which node** in the next layer the signal is going.

Input Data Shape and Output Layer Configuration

`X_train.shape`

- `(60000, 28, 28)` → This indicates that the dataset consists of **60,000 images**, each of **size 28×28 pixels**.
- These images are stored as **NumPy arrays** with shape `(28, 28)` for each sample.

Flatten Layer

- A **Flatten layer** is used to **convert higher-dimensional arrays into a 1D vector**.
- For example, each 28×28 image becomes a **784-dimensional vector** ($28 \times 28 = 784$), which can then be fed into dense layers.

Output Layer Configuration

When Using Softmax for Classification:

- If the output layer has **more than one node** (e.g., for multiclass classification), use the **Softmax activation function**.
- It converts raw scores into probabilities for each class.

Loss Functions in `model.compile()`

`sparse_categorical_crossentropy`

- Use this when labels are **integer-encoded** (e.g., `3` instead of `[0, 0, 0, 1, 0, 0, 0, 0, 0]`).
- **No need for one-hot encoding** of the target labels.

`categorical_crossentropy`

- Use this when labels are **one-hot encoded**.
- Requires converting class labels into one-hot vectors before training.

For Regression Problems

- When solving a **regression task**, your:
 - **Output layer** should contain **one node**.
 - **Activation function** should be `'linear'` .

In []: